

Apostila prática

Criação de Projeto no Quartus Prime

para o Kit DE10-Lite

Top-level design em VHDL com pinos configurados

Prof. Felipe Walter Dafico Pfrimer

Objetivo

Esta apostila apresenta um fluxo prático para criar um projeto no Quartus Prime Lite para o kit DE10-Lite, selecionar corretamente a placa e o dispositivo FPGA, criar um top-level design em VHDL e configurar os pinos físicos. O exemplo principal implementa um contador decimal de 00 a 99, com reset, carga paralela pelas chaves e exibição nos displays de sete segmentos.

19 de maio de 2026

Sumário

1	Materiais necessários	1
2	Informações do kit DE10-Lite	1
3	Criação do projeto no Quartus	2
4	Criação do arquivo VHDL	7
5	Top-level design em VHDL	9
6	Definição da entidade principal	12
7	Configuração dos pinos	13
8	Compilação do projeto	15
9	Gravação na placa	16
10	Erros comuns	20
11	Atividade proposta	20
12	Conclusão	20

1 Materiais necessários

Para acompanhar o roteiro, utilize:

- Kit FPGA Terasic DE10-Lite.
- Cabo USB conectado à porta USB-Blaster da placa.
- Quartus Prime Lite instalado.
- Arquivo VHDL do top-level design.
- Tabela de pinos da DE10-Lite ou arquivo `.qsf` com os pinos já atribuídos.

Checklist rápido antes de começar

- A placa está conectada pela porta USB-Blaster.
- O Quartus Prime Lite abre sem solicitar instalação adicional do dispositivo MAX 10.
- A pasta do projeto foi criada em um caminho simples, sem acentos e sem espaços.
- O nome escolhido para a entidade principal será repetido exatamente no arquivo VHDL.

2 Informações do kit DE10-Lite

O kit DE10-Lite utiliza um FPGA da família MAX 10. Para o projeto apresentado nesta apostila, a seleção do dispositivo deve seguir a Tabela 1.

Tabela 1: *Configurações principais do projeto para a DE10-Lite.*

Item	Configuração
Família	MAX 10
Dispositivo	10M50DAF484C6GES
Placa	MAX 10 DE10-Lite
Entidade de topo do exemplo	Contador_DE10_Lite
Linguagem usada na apostila	VHDL

O nome da entidade de topo em VHDL precisa coincidir com o nome definido no Quartus em *Top-Level Entity*. Neste exemplo, será usado `Contador_DE10_Lite`. Se houver diferença de maiúsculas, minúsculas ou sublinhados, o Quartus poderá não encontrar a entidade principal durante a compilação.

Atenção

O Quartus diferencia letras maiúsculas, minúsculas e sublinhados no nome da entidade. Se o projeto estiver configurado como `Contador_DE10_Lite`, a declaração VHDL deve usar exatamente esse nome.

3 Criação do projeto no Quartus

Ao abrir o Quartus Prime Lite, a tela inicial apresenta o botão *New Project Wizard*, indicado na Figura 1. Clique nele para iniciar a criação do projeto.

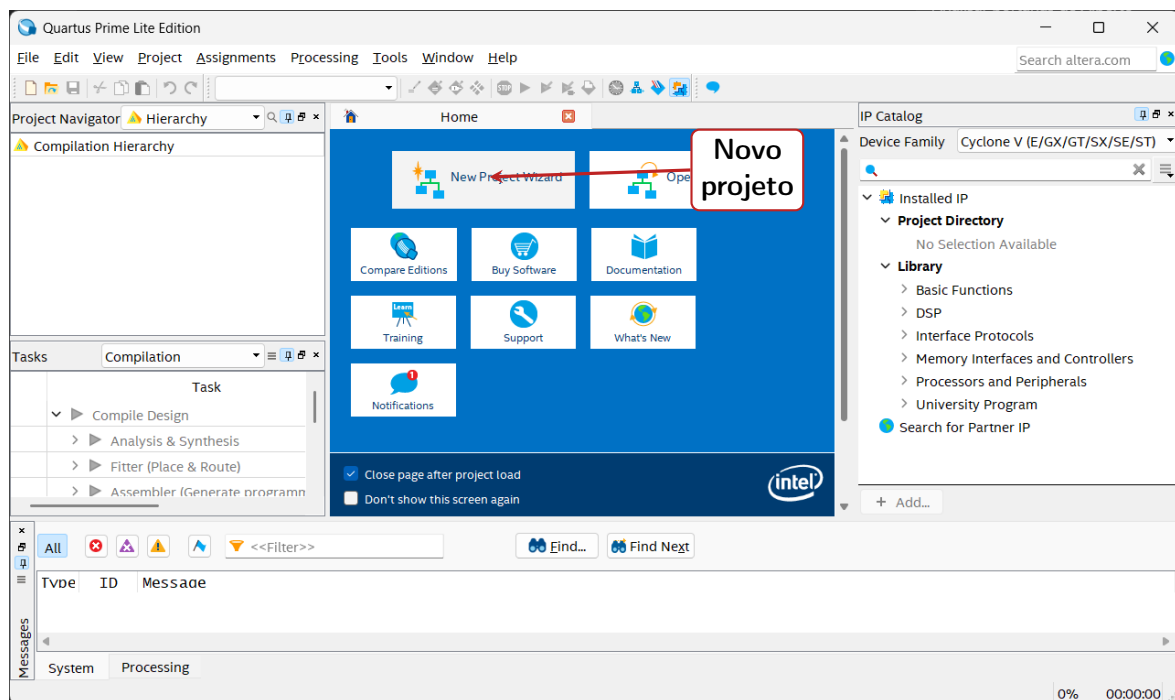


Figura 1: Tela inicial do Quartus Prime Lite.

A primeira janela do assistente é uma introdução ao fluxo de criação, como mostra a Figura 2. Clique em *Next* para avançar.

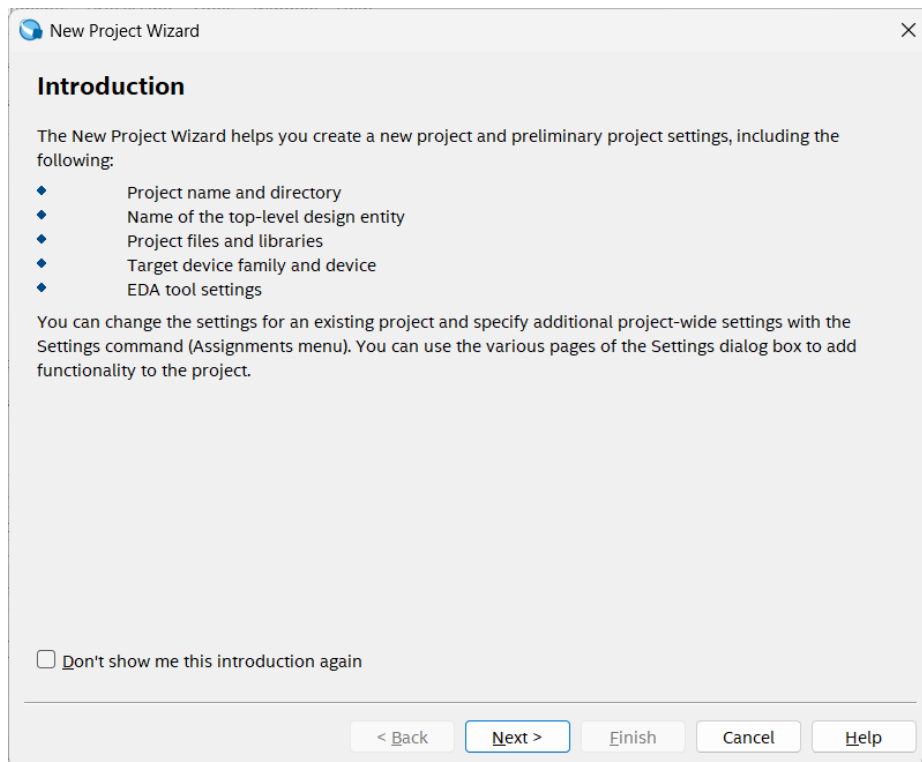


Figura 2: Introdução do assistente New Project Wizard.

Na etapa da Figura 3, escolha a pasta do projeto e informe o nome da entidade principal. O Código 1 mostra uma sugestão de preenchimento.

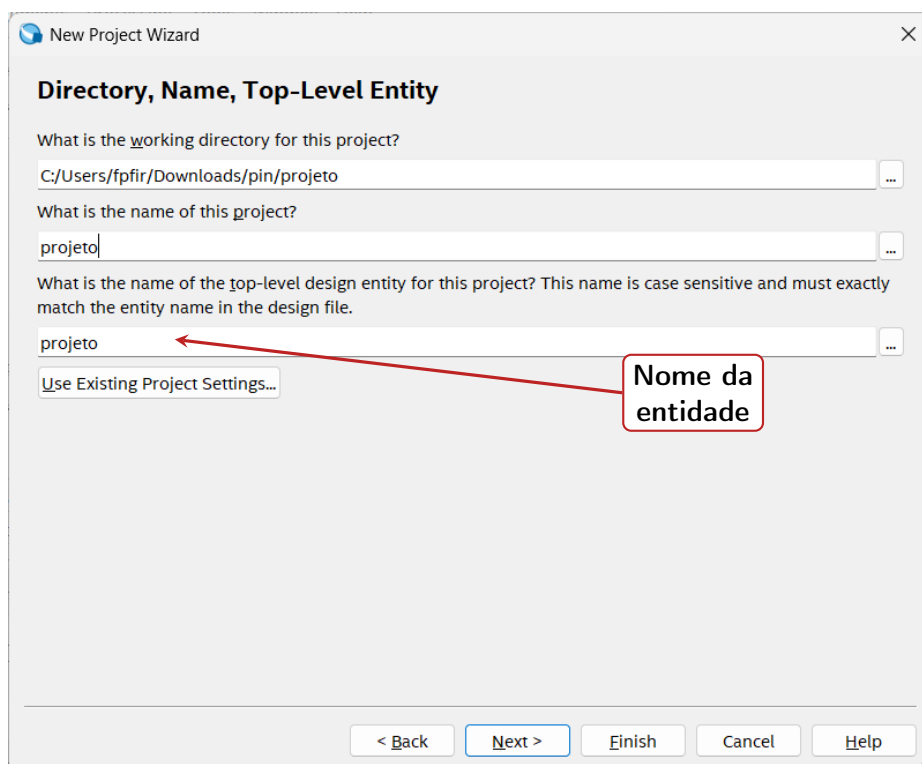


Figura 3: Definição do diretório, nome do projeto e entidade de topo.

```
1 Pasta do projeto: C:\Projetos\DE10_Lite
2 Nome do projeto: projeto
3 Top-level entity: Contador_DE10_Lite
```

Código 1: *Exemplo de organização inicial do projeto.*

Na etapa de tipo de projeto, selecione *Empty project*, conforme a Figura 4. Essa opção cria um projeto vazio, no qual o arquivo VHDL será adicionado manualmente.

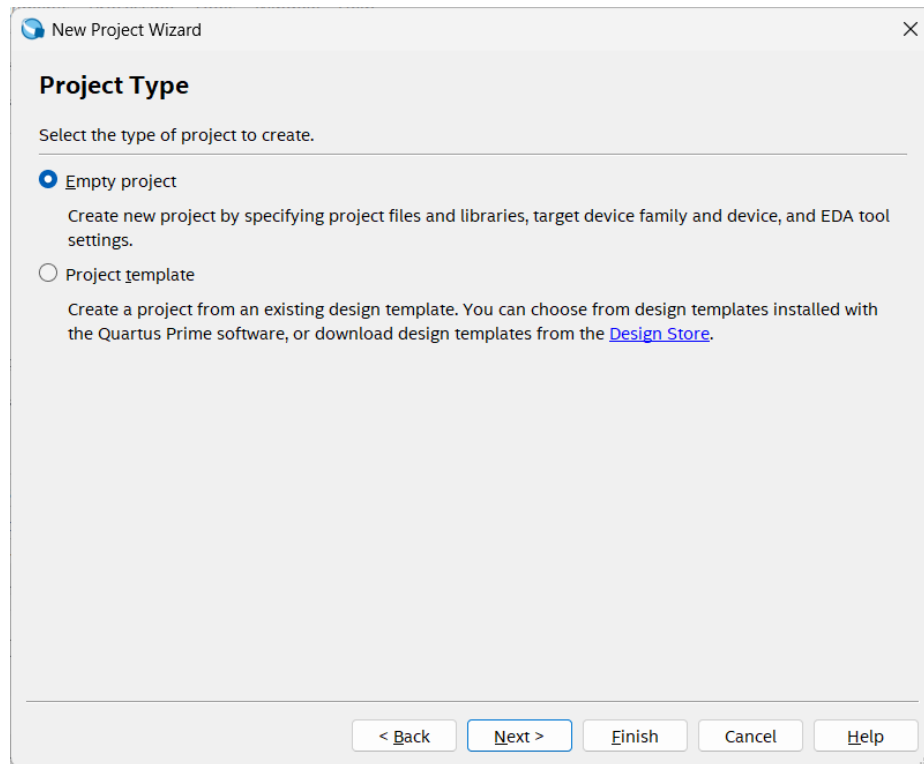


Figura 4: *Seleção do tipo de projeto.*

Caso já exista um arquivo VHDL pronto, adicione-o na tela da Figura 5. Se ainda não existir, avance sem adicionar arquivos e crie o VHDL depois.

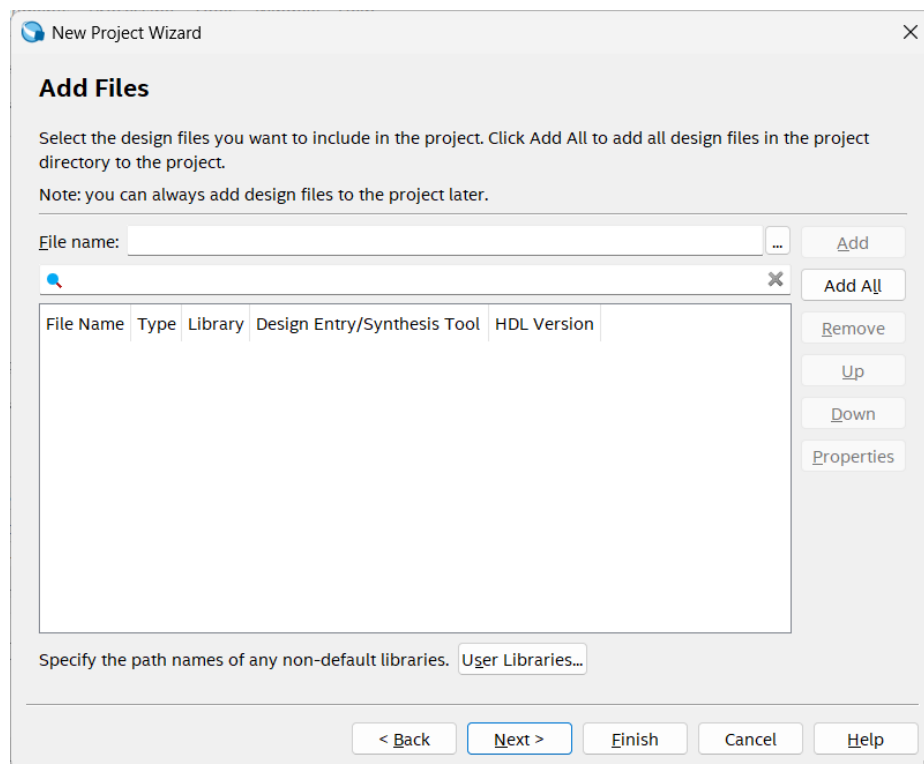


Figura 5: *Inclusão de arquivos existentes no projeto.*

Na etapa de dispositivo, a tela pode aparecer inicialmente com outra família selecionada, como na Figura 6. Selecione a família MAX 10 para trabalhar com a DE10-Lite e digite 10M50DAF484C6GES em *Name Filter*. No entanto, essa não é a melhor forma de configurar o projeto. Existe uma opção mais prática, que é selecionar a placa diretamente, como mostrado na próxima etapa.

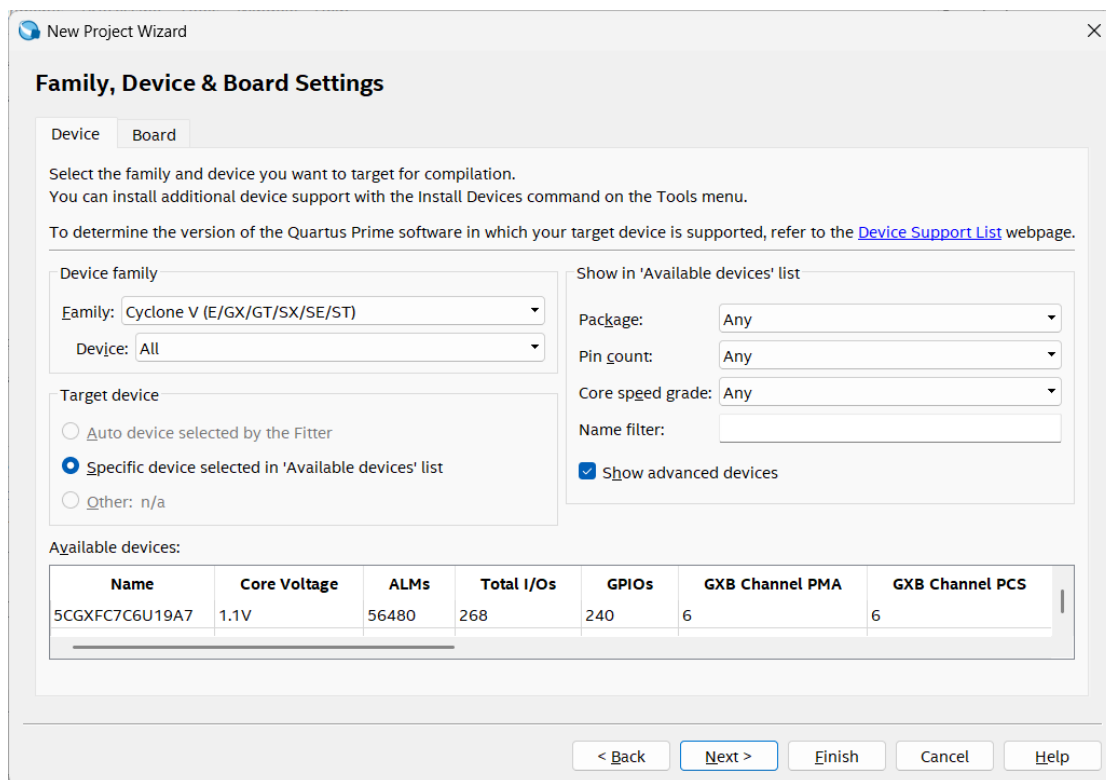


Figura 6: Tela de seleção de família, dispositivo e placa.

Alternativamente, na aba *Board*, selecione a placa *MAX 10 DE10-Lite*, como indicado na Figura 7. Essa tela também possui a opção *Create top-level design file*.

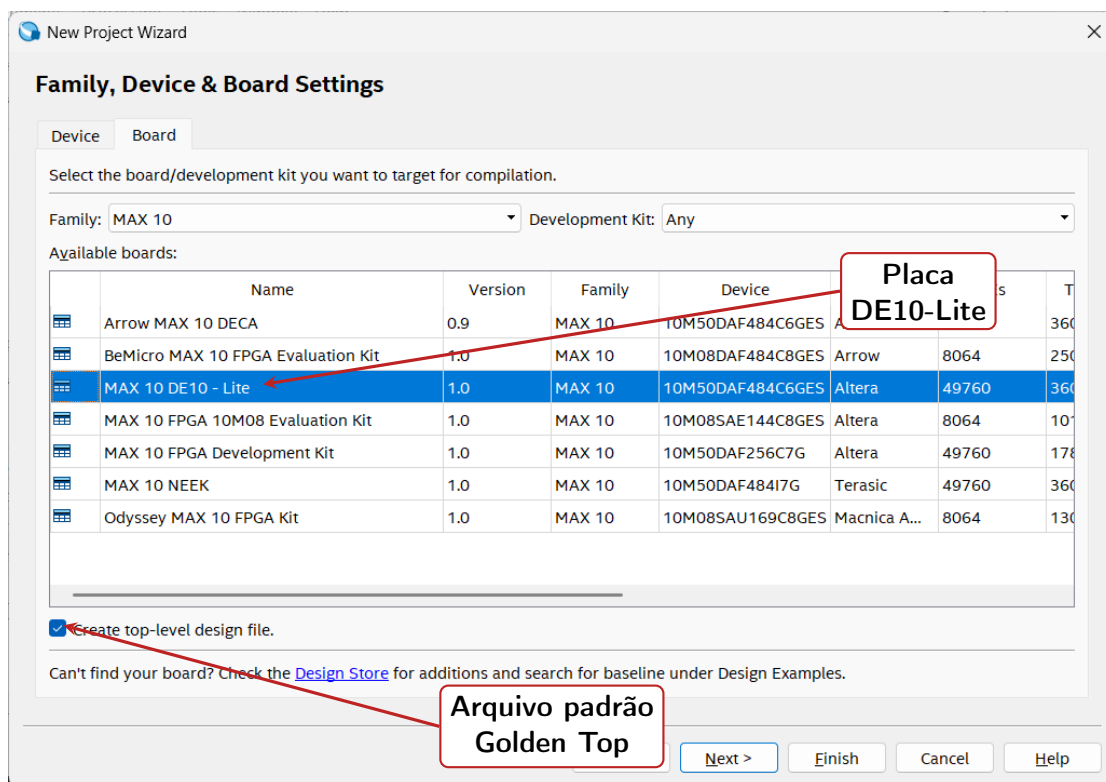


Figura 7: Seleção da placa e opção de criação automática do arquivo de topo.

Quando a opção *Create top-level design file* da Figura 7 fica marcada, o Quartus/Terasic pode criar automaticamente o arquivo `DE10_LITE_Golden_Top.v`. Nesta apostila, esse arquivo não será usado como descrição do circuito. Ele servirá apenas como referência para consultar nomes de portas da placa, como `SW`, `KEY`, `LEDR`, `HEX0`, `HEX1` e `MAX10_CLK1_50`. O circuito do aluno será escrito em VHDL, em um arquivo separado.

Um detalhe importante: se o top-level em VHDL usar exatamente os mesmos nomes de portas que aparecem no Golden Top e que já estão atribuídos no projeto, o Quartus consegue aproveitar as configurações de pinos já existentes. Nesse caso, não é necessário refazer manualmente essas atribuições no Pin Planner nem reescrevê-las no arquivo `.qsf`. A configuração manual só será necessária quando o projeto não tiver essas atribuições prontas ou quando os nomes das portas forem diferentes.

Papel do Golden Top

O arquivo `DE10_LITE_Golden_Top.v` pode ser útil como uma lista de nomes oficiais das portas da placa. Nesta apostila, ele não descreve o circuito e não precisa ser chamado pelo VHDL. O top-level real será o arquivo `Contador_DE10_Lite.vhd`.

4 Criação do arquivo VHDL

Com o projeto aberto, crie um novo arquivo em:

File > New > VHDL File

A Figura 8 mostra a janela de criação de arquivos do Quartus. Nela, selecione *VHDL File* dentro do grupo *Design Files* e clique em *OK*.

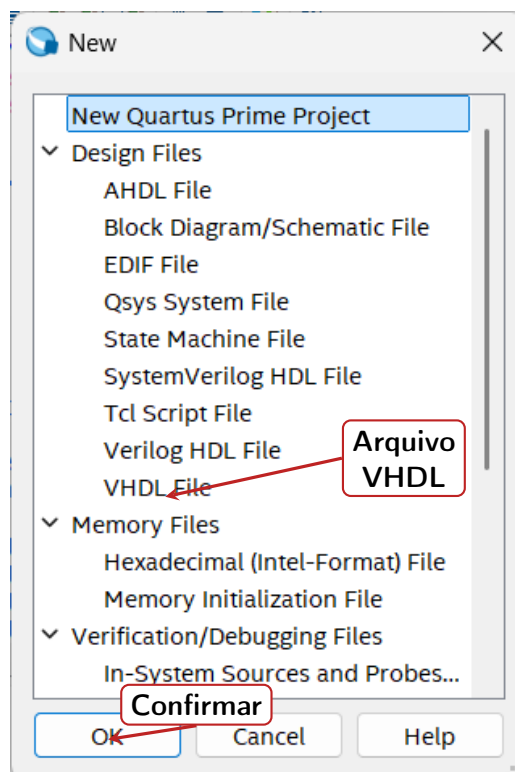


Figura 8: Seleção da opção *VHDL File* para criar o arquivo do contador.

Após criar o arquivo da Figura 8, salve-o como:

`Contador_DE10_Lite.vhd`

Depois de salvar o arquivo, adicione-o ao projeto se necessário. A Figura 9 mostra um projeto aberto após a criação automática do Golden Top. Note que o arquivo `DE10_LITE_Golden_Top.v` aparece na lista, mas ele será usado apenas para consulta.

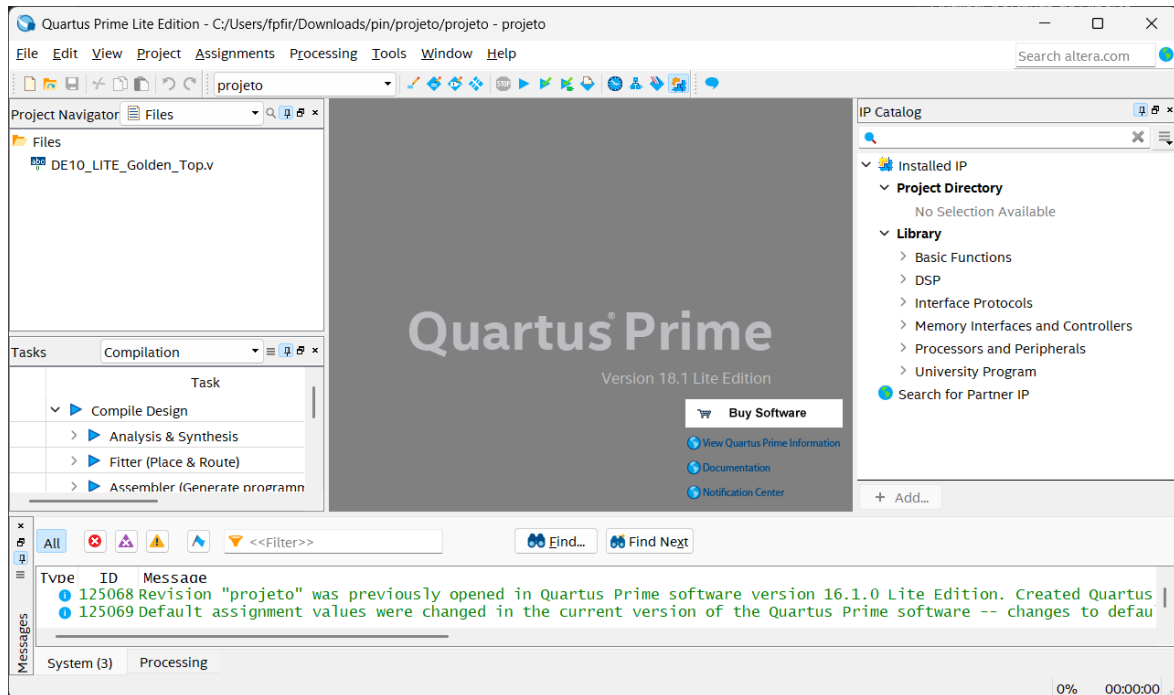


Figura 9: Projeto aberto com o arquivo Golden Top criado automaticamente.

A Figura 10 mostra o Golden Top aberto no editor. O trecho exibido é útil porque mostra nomes de portas da placa, por exemplo `HEX0`, `HEX1`, `KEY`, `LEDR` e `SW`. No VHDL do contador, não é necessário copiar todas essas portas; basta declarar as portas realmente usadas.

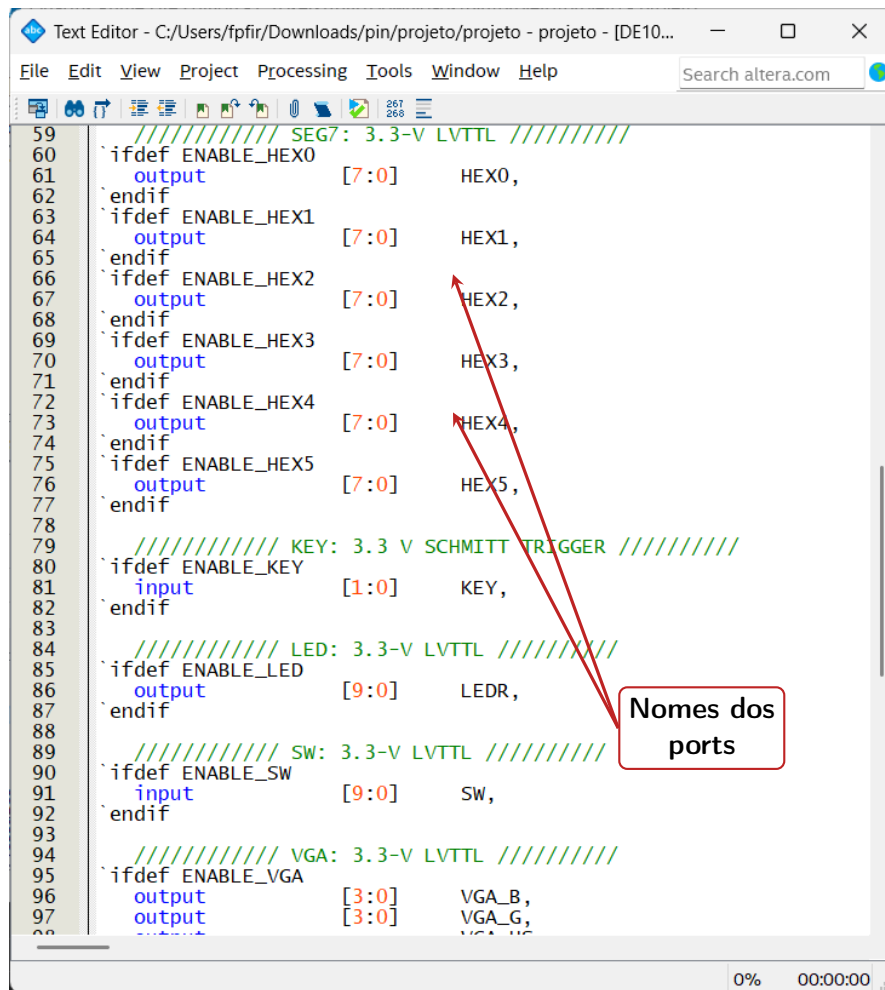


Figura 10: Arquivo Golden Top usado apenas como referência para os nomes das portas.

5 Top-level design em VHDL

O top-level design é a entidade principal do projeto. Ele declara quais sinais entram e saem do FPGA e permite que o Quartus relacione esses sinais aos pinos físicos da placa. Neste exemplo, serão usados os sinais da Tabela 2.

Tabela 2: Sinais usados pelo contador em VHDL.

Sinal	Direção	Função no circuito
MAX10_CLK1_50	Entrada	Clock de 50 MHz da placa.
KEY(0)	Entrada	Reset ativo em nível baixo.
KEY(1)	Entrada	Carga paralela ativa em nível baixo.
SW(7 downto 0)	Entrada	Valor BCD para carga paralela: dezenas em SW(7 downto 4) e unidades em SW(3 downto 0).
HEX0	Saída	Display de sete segmentos das unidades.
HEX1	Saída	Display de sete segmentos das dezenas.
LEDR	Saída	LEDs usados para visualizar as chaves.

Os botões KEY e os displays HEX da DE10-Lite são ativos em nível baixo. Isso significa que o botão pressionado gera valor lógico '0', e um segmento do display acende quando recebe '0'.

O Código 2 implementa um contador decimal de 00 a 99. O contador avança aproximadamente uma vez por segundo, usando o clock de 50 MHz da placa. O botão KEY(0) zera a contagem, enquanto KEY(1) carrega o valor informado pelas chaves SW(7 downto 0).

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity Contador_DE10_Lite is
6      port (
7          MAX10_CLK1_50 : in  std_logic;
8          KEY            : in  std_logic_vector(1 downto 0);
9          SW             : in  std_logic_vector(9 downto 0);
10         LEDR           : out std_logic_vector(9 downto 0);
11         HEX0           : out std_logic_vector(7 downto 0);
12         HEX1           : out std_logic_vector(7 downto 0)
13     );
14 end entity Contador_DE10_Lite;
15
16 architecture rtl of Contador_DE10_Lite is
17     constant DIVISOR_1HZ : natural := 50000000;
18
19     signal div_count : natural range 0 to DIVISOR_1HZ - 1 := 0;
20     signal tick_1hz  : std_logic := '0';
21     signal unidade   : unsigned(3 downto 0) := (others => '0');
22     signal dezena    : unsigned(3 downto 0) := (others => '0');
23
24     function seg7(bcd : unsigned(3 downto 0)) return
25         std_logic_vector is
26     begin

```

```

26     case bcd is
27         when "0000" => return "11000000"; -- 0
28         when "0001" => return "11111001"; -- 1
29         when "0010" => return "10100100"; -- 2
30         when "0011" => return "10110000"; -- 3
31         when "0100" => return "10011001"; -- 4
32         when "0101" => return "10010010"; -- 5
33         when "0110" => return "10000010"; -- 6
34         when "0111" => return "11111000"; -- 7
35         when "1000" => return "10000000"; -- 8
36         when "1001" => return "10010000"; -- 9
37         when others => return "11111111"; -- apagado
38     end case;
39 end function;
40 begin
41     LEDR <= SW;
42
43     process (MAX10_CLK1_50)
44     begin
45         if rising_edge(MAX10_CLK1_50) then
46             if div_count = DIVISOR_1HZ - 1 then
47                 div_count <= 0;
48                 tick_1hz <= '1';
49             else
50                 div_count <= div_count + 1;
51                 tick_1hz <= '0';
52             end if;
53         end if;
54     end process;
55
56     process (MAX10_CLK1_50)
57         variable carga_unidade : unsigned(3 downto 0);
58         variable carga_dezena  : unsigned(3 downto 0);
59     begin
60         if rising_edge(MAX10_CLK1_50) then
61             carga_unidade := unsigned(SW(3 downto 0));
62             carga_dezena  := unsigned(SW(7 downto 4));
63
64             if KEY(0) = '0' then
65                 unidade <= (others => '0');
66                 dezena  <= (others => '0');
67             elsif KEY(1) = '0' then
68                 if carga_unidade <= to_unsigned(9, 4) and
69                    carga_dezena <= to_unsigned(9, 4) then
70                     unidade <= carga_unidade;
71                     dezena  <= carga_dezena;
72                 else
73                     unidade <= (others => '0');
74                     dezena  <= (others => '0');
75                 end if;
76             elsif tick_1hz = '1' then

```

```

77         if unidade = to_unsigned(9, 4) then
78             unidade <= (others => '0');
79
80             if dezena = to_unsigned(9, 4) then
81                 dezena <= (others => '0');
82             else
83                 dezena <= dezena + 1;
84             end if;
85         else
86             unidade <= unidade + 1;
87         end if;
88     end if;
89 end if;
90 end process;
91
92 HEX0 <= seg7(unidade);
93 HEX1 <= seg7(dezena);
94 end architecture rtl;

```

Código 2: Top-level design em VHDL: contador decimal com reset e carga paralela.

6 Definição da entidade principal

Depois de salvar o arquivo VHDL, confirme se ele está definido como top-level:

Project > Set as Top-Level Entity

Uma forma prática de fazer isso é clicar com o botão direito sobre o arquivo VHDL e escolher *Set as Top-Level Entity*, como mostra a Figura 11. Também é possível verificar essa configuração em:

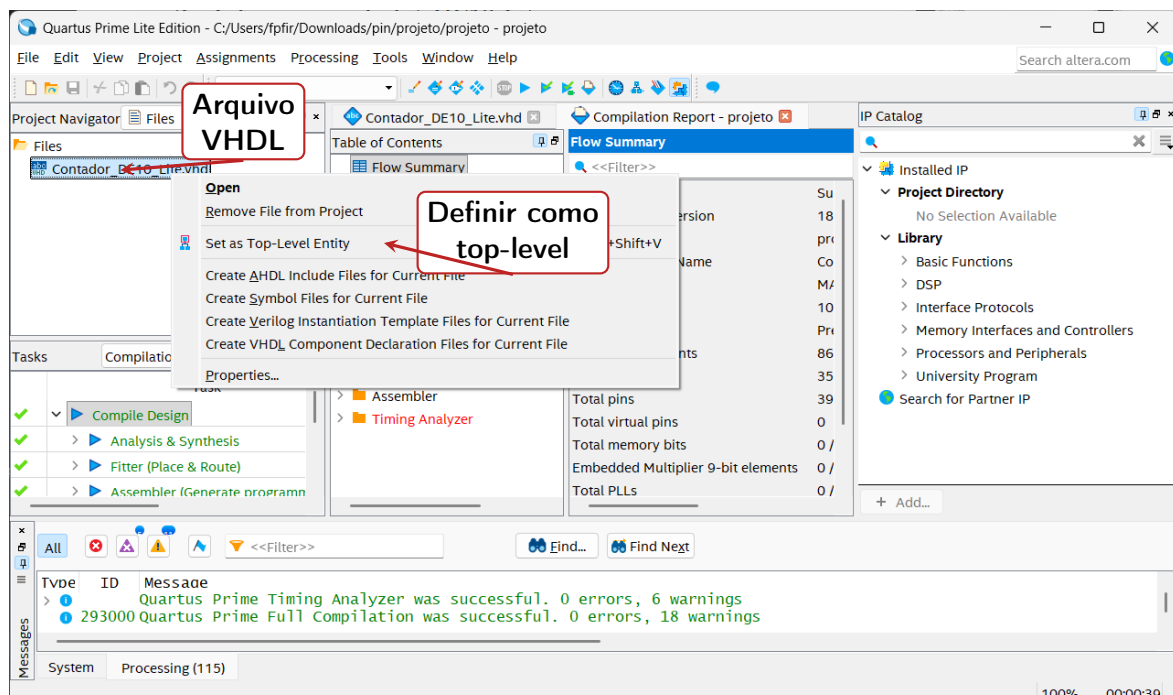


Figura 11: Definição do arquivo VHDL como entidade principal do projeto.

Além do menu de contexto da Figura 11, também é possível verificar essa configuração em:

Assignments > Settings > General > Top-level entity

O nome configurado deve ser o mesmo da entidade do Código 2: Contador_DE10_Lite.

7 Configuração dos pinos

A configuração dos pinos pode ser feita pela interface gráfica, usando o *Pin Planner*, ou diretamente pelo arquivo *.qsf*. A Figura 12 mostra o Pin Planner com atribuições de displays de sete segmentos.

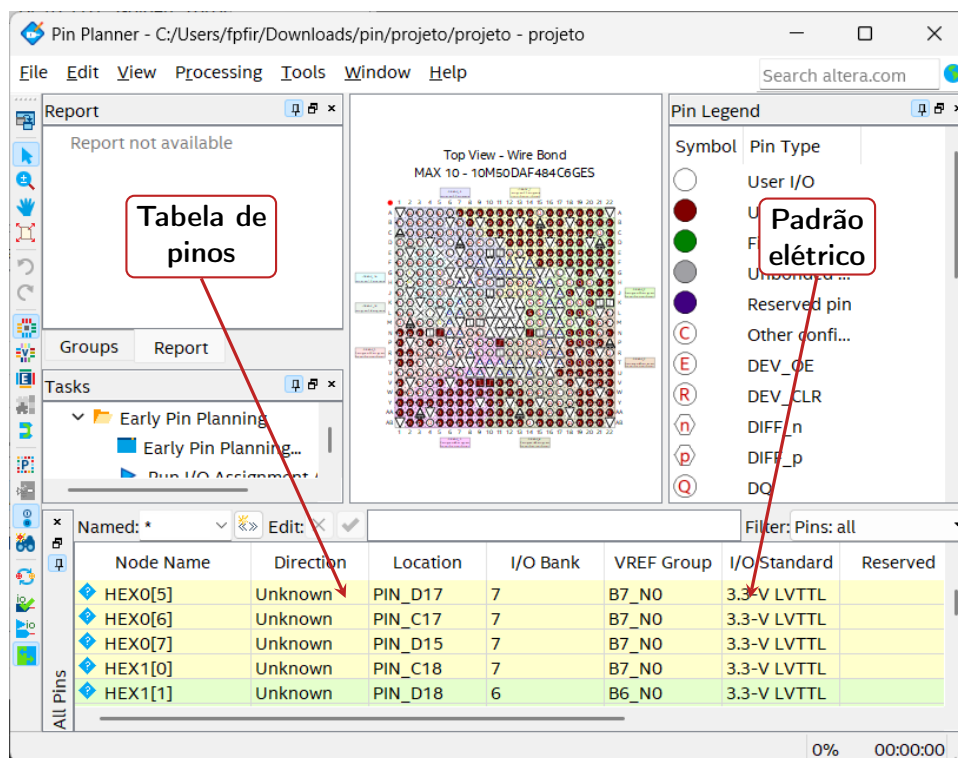


Figura 12: *Pin Planner* usado para associar sinais aos pinos físicos.

No Pin Planner, cada sinal do top-level deve receber uma localização física, como PIN_A8, e um padrão elétrico de entrada e saída, como 3.3-V LVTTTL. Para os botões KEY, utiliza-se 3.3 V SCHMITT TRIGGER. A Tabela 3 lista os pinos necessários para o contador.

Se o projeto foi criado a partir da opção da Figura 7 e o VHDL manteve os mesmos nomes do Golden Top, como MAX10_CLK1_50, KEY, SW, LEDR, HEX0 e HEX1, as linhas correspondentes do *.qsf* já podem estar prontas. Nessa situação, a Tabela 3 funciona como conferência, não como uma etapa obrigatória de digitação.

Sinais usados no exemplo

MAX10_CLK1_50	clock de 50 MHz da placa
KEY(0)	reset do contador
KEY(1)	carga paralela do valor das chaves
SW(7 downto 0)	valor BCD carregado no contador
LEDR	indicação visual do valor das chaves
HEX0 e HEX1	displays de sete segmentos usados para mostrar a contagem

Tabela 3: *Atribuições de pinos usadas no exemplo do contador.*

Sinal	Pino físico	Padrão de I/O
MAX10_CLK1_50	PIN_P11	3.3-V LVTTL
SW[0]	PIN_C10	3.3-V LVTTL
SW[1]	PIN_C11	3.3-V LVTTL
SW[2]	PIN_D12	3.3-V LVTTL
SW[3]	PIN_C12	3.3-V LVTTL
SW[4]	PIN_A12	3.3-V LVTTL
SW[5]	PIN_B12	3.3-V LVTTL
SW[6]	PIN_A13	3.3-V LVTTL
SW[7]	PIN_A14	3.3-V LVTTL
SW[8]	PIN_B14	3.3-V LVTTL
SW[9]	PIN_F15	3.3-V LVTTL
KEY[0]	PIN_B8	3.3 V SCHMITT TRIGGER
KEY[1]	PIN_A7	3.3 V SCHMITT TRIGGER
LEDR[0]	PIN_A8	3.3-V LVTTL
LEDR[1]	PIN_A9	3.3-V LVTTL
LEDR[2]	PIN_A10	3.3-V LVTTL
LEDR[3]	PIN_B10	3.3-V LVTTL
LEDR[4]	PIN_D13	3.3-V LVTTL
LEDR[5]	PIN_C13	3.3-V LVTTL
LEDR[6]	PIN_E14	3.3-V LVTTL
LEDR[7]	PIN_D14	3.3-V LVTTL
LEDR[8]	PIN_A11	3.3-V LVTTL
LEDR[9]	PIN_B11	3.3-V LVTTL
HEX0[0]	PIN_C14	3.3-V LVTTL
HEX0[1]	PIN_E15	3.3-V LVTTL
HEX0[2]	PIN_C15	3.3-V LVTTL
HEX0[3]	PIN_C16	3.3-V LVTTL
HEX0[4]	PIN_E16	3.3-V LVTTL
HEX0[5]	PIN_D17	3.3-V LVTTL

Tabela 3: Atribuições de pinos usadas no exemplo do contador (continuação).

Sinal	Pino físico	Padrão de I/O
HEX0[6]	PIN_C17	3.3-V LVTTL
HEX0[7]	PIN_D15	3.3-V LVTTL
HEX1[0]	PIN_C18	3.3-V LVTTL
HEX1[1]	PIN_D18	3.3-V LVTTL
HEX1[2]	PIN_E18	3.3-V LVTTL
HEX1[3]	PIN_B16	3.3-V LVTTL
HEX1[4]	PIN_A17	3.3-V LVTTL
HEX1[5]	PIN_A18	3.3-V LVTTL
HEX1[6]	PIN_B17	3.3-V LVTTL
HEX1[7]	PIN_A16	3.3-V LVTTL

Também existe a opção de trabalhar diretamente com o arquivo `.qsf`, que é o arquivo de configurações do projeto do Quartus. Segundo a documentação da Intel, o *Quartus Prime Settings File* armazena as configurações e atribuições do projeto, incluindo atribuições feitas por janelas do Quartus ou por comandos Tcl. Nesta apostila, essa alternativa é mencionada apenas como referência; o fluxo recomendado para o aluno é conferir ou ajustar os pinos pelo Pin Planner. Para mais detalhes, consulte a definição oficial da Intel em https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_qsf.htm.

8 Compilação do projeto

Após salvar o VHDL e configurar os pinos, execute:

Processing > Start Compilation

A Figura 13 mostra o Quartus durante a compilação. A janela *Tasks* permite acompanhar as etapas do fluxo, como análise e síntese, posicionamento e roteamento, montagem e análise temporal. Se a compilação terminar sem erros, o Quartus gerará o arquivo de programação na pasta `output_files`. Normalmente, o arquivo utilizado para gravação possui extensão `.sof`.

Após a compilação

Antes de abrir o Programmer, confira três pontos: a janela de mensagens deve indicar ausência de erros, o arquivo `.sof` deve existir em `output_files` e o top-level compilado deve ser o `Contador_DE10_Lite`. Avisos podem existir, mas erros impedem a gravação correta do circuito.

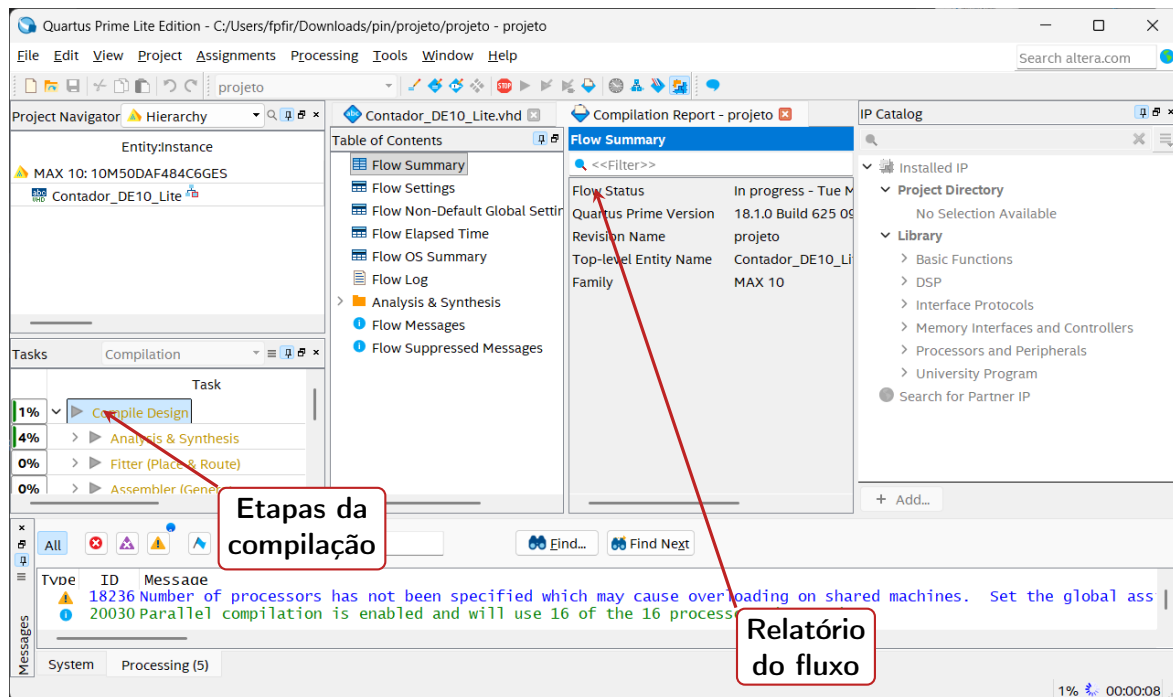


Figura 13: Compilação em andamento no Quartus Prime.

9 Gravação na placa

Conecte a DE10-Lite ao computador pelo cabo USB e abra:

Tools > Programmer

Na janela do Programmer, mostrada na Figura 14, o arquivo .sof deve estar carregado e a opção *Program/Configure* deve estar marcada. Se o campo de hardware aparecer como *No Hardware*, clique em *Hardware Setup*.

Atenção

Se o campo de hardware continuar como *No Hardware*, verifique o cabo USB, a alimentação da placa, o driver USB-Blaster e a seleção feita em *Hardware Setup*. Sem o USB-Blaster selecionado, o botão *Start* não grava o FPGA.

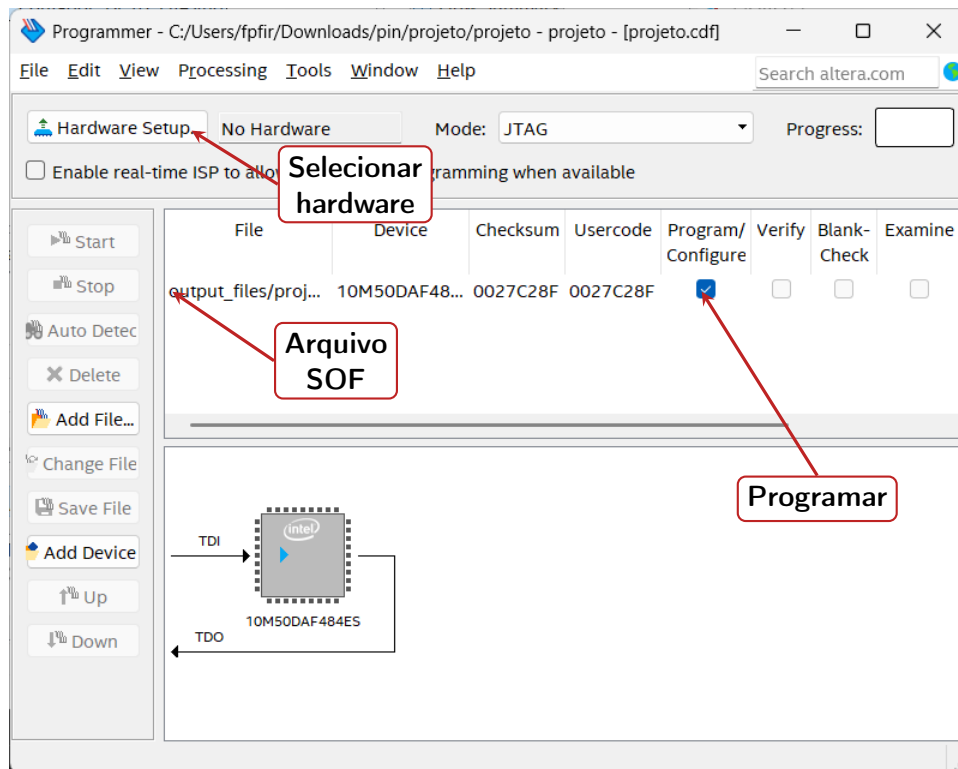


Figura 14: *Programmer aberto antes da seleção do hardware USB-Blaster.*

Na tela de configuração de hardware, a Figura 15 mostra a situação em que nenhum gravador está selecionado. Após conectar a placa e selecionar o gravador, o campo deve indicar USB-Blaster [USB-0], como na Figura 16.

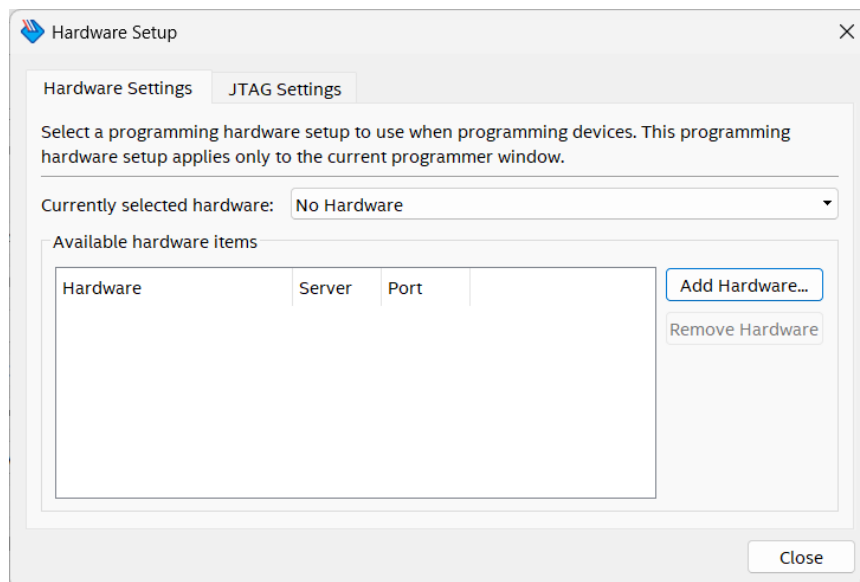


Figura 15: *Janela de configuração sem hardware selecionado.*

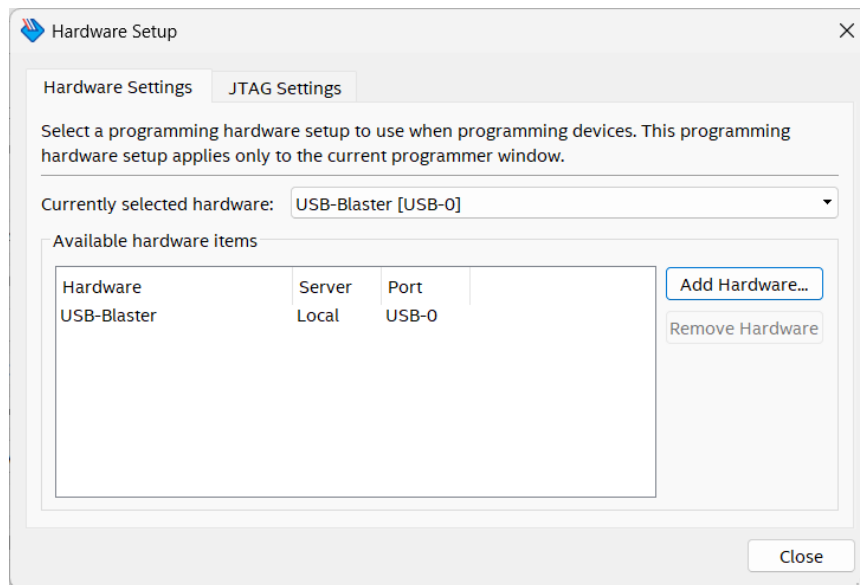


Figura 16: Janela de configuração com USB-Blaster selecionado.

Depois que o USB-Blaster estiver selecionado, retorne ao Programmer. A Figura 17 mostra a tela pronta para gravação; clique em *Start*. Ao final, o progresso deve chegar a 100% (*Successful*), como mostrado na Figura 18.

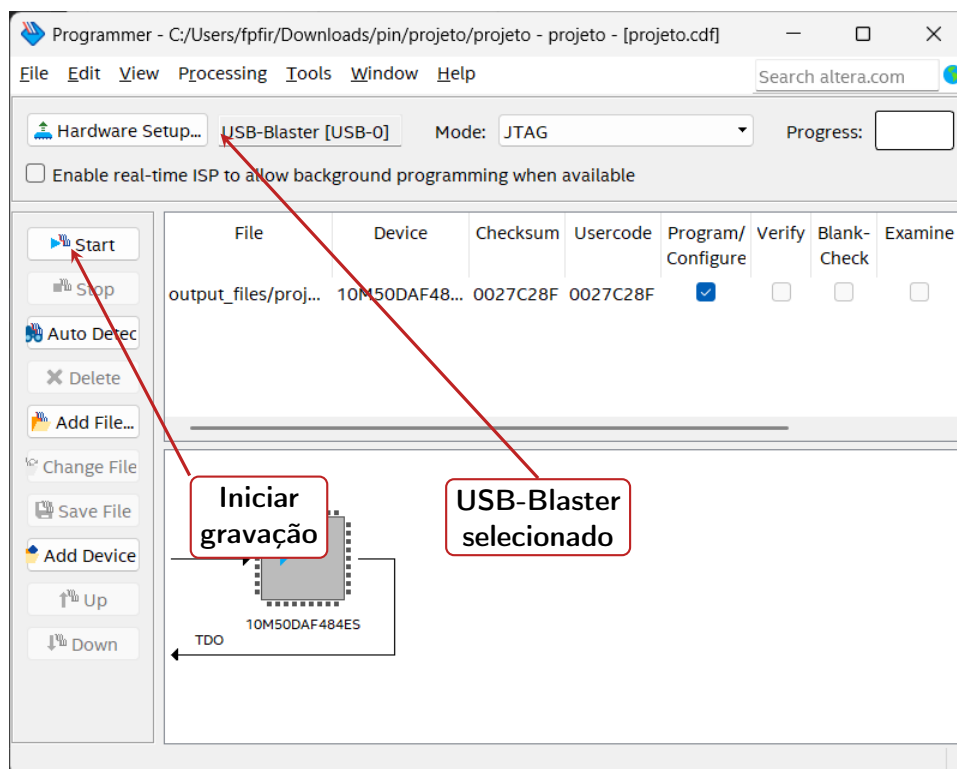


Figura 17: Programmer pronto para iniciar a gravação.

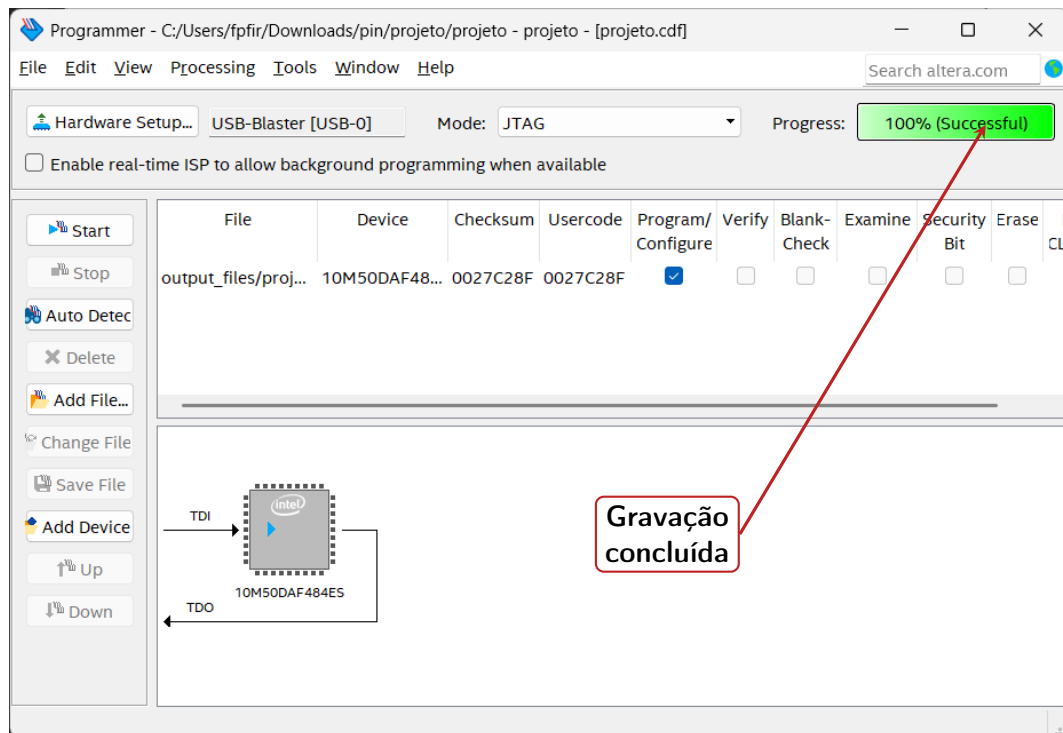


Figura 18: Gravação concluída com sucesso no kit DE10-Lite.

Para testar:

- Pressione KEY(0) para zerar a contagem.
- Configure um valor BCD nas chaves SW(7 down to 0).
- Pressione KEY(1) para carregar o valor.
- Observe a contagem nos displays HEX1 e HEX0, como na Figura 19.

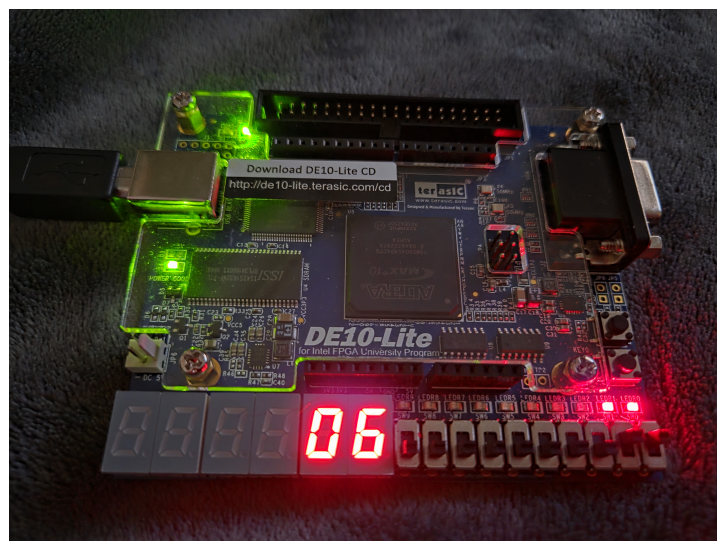


Figura 19: Kit DE10-Lite programado e exibindo a contagem nos displays de sete segmentos.

Leitura rápida do teste

Com KEY(0) o valor volta para 00. Com KEY(1) o contador carrega o valor BCD definido em SW(7 downto 0). Depois da carga, os displays HEX1 e HEX0 devem avançar a contagem decimal.

10 Erros comuns

A Tabela 4 resume problemas frequentes durante a criação, compilação e teste do projeto.

Tabela 4: *Erros comuns durante a criação do projeto.*

Problema	Possível solução
O Quartus não encontra a entidade principal	Verifique se o nome da entidade VHDL é igual ao campo <i>Top-level entity</i> .
Erro de pino não atribuído	Abra o Pin Planner ou confira se o sinal possui <code>set_location_assignment</code> no arquivo .qsf.
Erro de padrão elétrico	Confira se cada sinal possui <code>IO_STANDARD</code> . Para chaves, clock, LEDs e displays, use <code>3.3-V LVTTL</code> ; para botões, use <code>3.3 V SCHMITT TRIGGER</code> .
Display de sete segmentos parece invertido	Lembre-se de que os segmentos são ativos em nível baixo: 0 acende e 1 apaga.
Carga paralela não mostra o valor esperado	Confira se as chaves estão em BCD válido: unidades em SW(3 downto 0) e dezenas em SW(7 downto 4), cada grupo de 0 a 9.

11 Atividade proposta

Modifique o contador para:

- usar SW(9) como habilitação da contagem;
- manter o valor congelado quando SW(9) = '0';
- piscar LEDR(9) a cada pulso de um segundo.

Desafio extra

Depois de concluir a atividade, altere o projeto para contar de 00 a 59, como em um cronômetro simples. Essa mudança exige limitar as unidades a 9 e as dezenas a 5.

12 Conclusão

Neste roteiro, foi criado um projeto no Quartus Prime Lite para a placa DE10-Lite, selecionando o dispositivo correto, definindo um top-level design próprio em VHDL e associando os sinais lógicos aos pinos físicos do FPGA. O arquivo `DE10_LITE_Golden_Top.v`, quando gerado, foi usado apenas como referência para os nomes das portas da placa. Esse procedi-

mento é a base para projetos digitais maiores, pois todo circuito implementado na DE10-Lite depende da correta relação entre o código HDL e os recursos físicos do kit.